



Data and Artificial Intelligence

Cyber Shujaa Program

Week 10 Project Report Deep Learning

Student Name: Amidu Dabor

Student ID: CS-DA01-25031

Date: 21 July, 2025

TABLE OF CONTENTS

1.0	INTRODUCTION	1
2.0	TASK COMPLETION	1
2.1	LOADING AND EXPLORING THE DATASET	1
2.2	DATA PREPARATION AND PREPROCESSING	2
2.3	MODEL IMPLEMENTATION.....	3
2.4	MODEL EVALUATION	4
2.5	TRAINING WITH EARLY STOPPING	6
2.6	SAVING AND RELOADING THE MODEL	6
2.7	PUBLISHING AND SHARING NOTEBOOK.....	7
3.0	PORTFOLIO WEBSITE	7
4.0	CONCLUSION.....	7
5.0	REFERENCES.....	8

1.0 INTRODUCTION

This project focuses on developing a deep learning model to classify handwritten digits using the MNIST dataset. The solution walks through the complete supervised learning pipeline using TensorFlow and Keras. The objective is to build, train, evaluate, and save a robust neural network capable of recognizing digits (0 - 9) from grayscale images of 28 x 28 pixels.

2.0 TASK COMPLETION

2.1 Loading and Exploring the Dataset

The MNIST dataset was loaded using `tensorflow.keras.datasets.mnist`. It contains 60,000 training images and 10,000 test images.

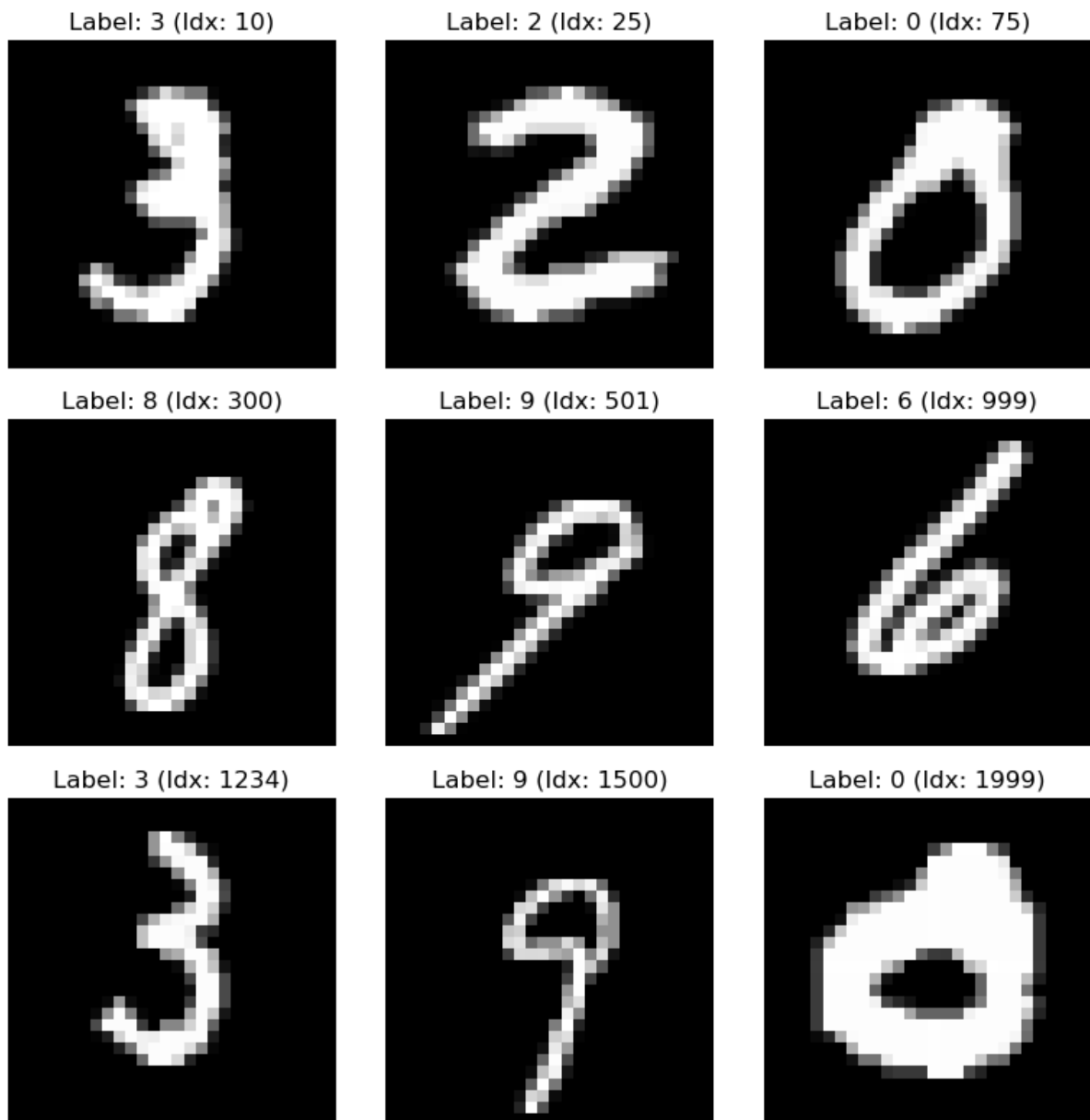
- Each image is a 28 x 28 grayscale pixel grid.
- Labels represent digits (0 - 9).

```
1 # Load and preprocess data
2 (train_X, train_y, train_y_cat), (test_X, test_y, test_y_cat) = classifier.load_and_preprocess_data()
3
4 print("Shape of x_train:", (train_X, train_y, train_y_cat)[0].shape, "| Shape of y_train:", (train_X, train_y, train_y_cat)[2].shape)
5 print("Shape of x_test:", (test_X, test_y, test_y_cat)[0].shape, "| Shape of y_test:", (test_X, test_y, test_y_cat)[2].shape)
```

✓ 0.2s Python

Shape of x_train: (60000, 28, 28) | Shape of y_train: (60000, 10)
Shape of x_test: (10000, 28, 28) | Shape of y_test: (10000, 10)

Additionally, **9 random images** were visualized using Matplotlib to confirm proper loading and labeling.



2.2 Data Preparation and Preprocessing

Key preprocessing steps included:

- **Normalization:** Pixel values were scaled from $[0, 255]$ to $[0, 1]$.
- **One-Hot Encoding:** Labels were converted into categorical format using `to_categorical`.
- **Train/Validation Split:** 10% of the training data was used for validation.

```

10 def load_and_preprocess_data(self) -> Tuple[Tuple[np.ndarray, np.ndarray], Tuple[np.ndarray, np.ndarray]]:
11     (X_train, y_train), (X_test, y_test) = mnist.load_data()
12     X_train, X_test = X_train / 255.0, X_test / 255.0
13     y_train_cat = to_categorical(y_train, 10)
14     y_test_cat = to_categorical(y_test, 10)
15
16     return (X_train, y_train, y_train_cat), (X_test, y_test, y_test_cat)
17

```

```

1 # Load and preprocess data
2 (train_X, train_y, train_y_cat), (test_X, test_y, test_y_cat) = classifier.load_and_preprocess_data()
3
4 print("Shape of x_train:", (train_X, train_y, train_y_cat)[0].shape, "| Shape of y_train:", (train_X, train_y, train_y_cat)[2].shape)
5 print("Shape of x_test:", (test_X, test_y, test_y_cat)[0].shape, "| Shape of y_test:", (test_X, test_y, test_y_cat)[2].shape)

```

✓ 0.2s Python

Shape of x_train: (60000, 28, 28) | Shape of y_train: (60000, 10)
Shape of x_test: (10000, 28, 28) | Shape of y_test: (10000, 10)

2.3 Model Implementation

A deep learning model was built using the `Sequential` API in Keras. The architecture included:

- Flatten input layer
- Two hidden `Dense` layers with **128** and **64** neurons using **ReLU** activation
- Dropout layers with a rate of **0.3** after each dense layer for regularization
- Dense output layer with 10 neurons using `softmax` activation

```

4. Build and compile the ANN model

```

```

1 # Build model
2 classifier.build_model()
3 # Compile model
4 classifier.compile_model()

```

[26] ✓ 0.0s Python

`/opt/anaconda3/envs/venv_py311/lib/python3.11/site-packages/keras/src/layers/resizing/flattten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to the 'Flatten' layer. It is not supported. Please use the 'input_shape'/'input_dim' argument to the 'Input' layer instead.`

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_6 (Dense)	(None, 128)	100,480
dropout_4 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

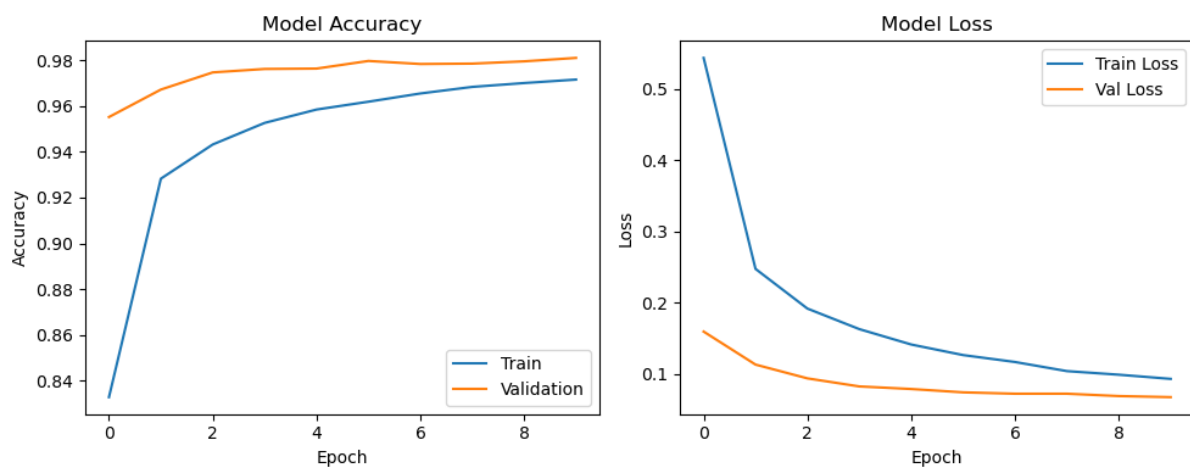
Non-trainable params: 0 (0.00 B)

The model was compiled using:

- **Optimizer:** Adam
- **Loss Function:** Categorical Crossentropy
- **Metrics:** Accuracy

2.4 Model Evaluation

After training the model (for 10 epochs), we evaluated it on the **test set**.



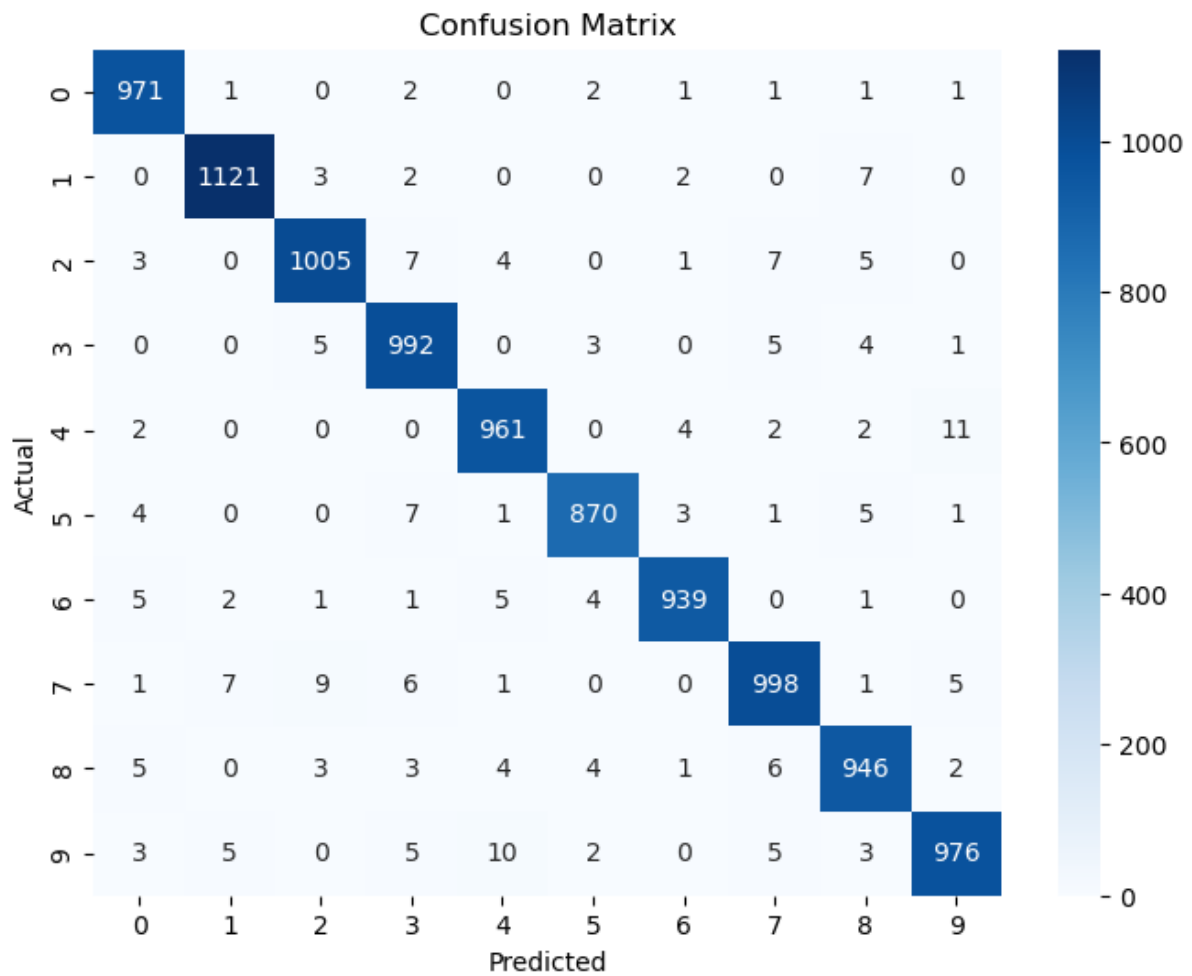
```
Model loaded from mnist_ann_model.keras
313/313 ————— 0s 625us/step - accuracy: 0.9746 - loss: 0.0873

Test Accuracy: 0.9779

0.9779000282287598
```

Test Accuracy: Final test accuracy was printed and confirmed.

- **Confusion Matrix** was plotted using `seaborn.heatmap` to visualize classification performance.
- **Classification Report** was generated using `sklearn.metrics.classification_report` to measure precision, recall, and F1-score.



```

1 # Report
2 classifier.report_results(test_X, test_y_cat, test_y)
[47] ✓ 0.3s
... 313/313 0s 411us/step

Classification Report:

      precision    recall  f1-score   support

     0       0.98       0.99       0.98        980
     1       0.99       0.99       0.99       1135
     2       0.98       0.97       0.98       1032
     3       0.97       0.98       0.97       1010
     4       0.97       0.98       0.98        982
     5       0.98       0.98       0.98        892
     6       0.99       0.98       0.98        958
     7       0.97       0.97       0.97       1028
     8       0.97       0.97       0.97        974
     9       0.98       0.97       0.97       1009

 accuracy          0.98       10000
 macro avg         0.98       0.98       0.98       10000
 weighted avg      0.98       0.98       0.98       10000

```

2.5 Training with Early Stopping

To improve model generalization and prevent overfitting, an alternative training strategy was implemented using **early stopping**.

Implementation:

The training leveraged the `EarlyStopping` callback from Keras:

- `monitor='val_loss'`: Monitors the validation loss during training.
- `patience=3`: Training stops if the validation loss does not improve for 3 consecutive epochs.

This allows the model to automatically halt training once it stops learning, thus avoiding unnecessary epochs and reducing overfitting.

```
1 # Train model - early stopping applied
2 classifier.train_with_early_stopping(train_X, train_y_cat)
#4] ✓ 8.2s
```

Epoch	422/422	1s	2ms/step	accuracy	loss	val_accuracy	val_loss
Epoch 1/10	422/422	1s	2ms/step	0.7088	0.9001	0.9552	0.1597
Epoch 2/10	422/422	1s	2ms/step	0.9232	0.2646	0.9672	0.1133
Epoch 3/10	422/422	1s	2ms/step	0.9409	0.2009	0.9747	0.0940
Epoch 4/10	422/422	1s	2ms/step	0.9517	0.1652	0.9762	0.0827
Epoch 5/10	422/422	1s	2ms/step	0.9580	0.1423	0.9763	0.0791
Epoch 6/10	422/422	1s	2ms/step	0.9608	0.1277	0.9797	0.0744
Epoch 7/10	422/422	1s	2ms/step	0.9654	0.1140	0.9783	0.0725
Epoch 8/10	422/422	1s	2ms/step	0.9671	0.1058	0.9785	0.0724
Epoch 9/10	422/422	1s	2ms/step	0.9704	0.0961	0.9795	0.0691
Epoch 10/10	422/422	1s	2ms/step	0.9731	0.0892	0.9810	0.0676

2.6 Saving and Reloading the Model

The trained model was saved using Keras' native `.h5` format:

10. Save model

```
1 # Save model
2 classifier.save_model()
```

[48] ✓ 0.0s

...

Model saved to mnist_ann_model.keras

11. Load and re-evaluate performance

```
1 # Load model
2 classifier.load_model()
3 # Re-evaluate performance
4 classifier.evaluate(test_X, test_y_cat)
```

[49] ✓ 0.4s

...

Model loaded from mnist_ann_model.keras

313/313 ————— 0s 625us/step - accuracy: 0.9746 - loss: 0.0873

Test Accuracy: 0.9779

... 0.9779000282287598

2.7 Publishing and Sharing Notebook

The final notebook was uploaded to Google Drive (VS Code Notebook) and made publicly accessible.

 VS Code Notebook Link:

[<https://drive.google.com/file/d/1jPGZzzgONZbs9iT26LsNNqmrFtu-0Uu3/view?usp=sharing>]

3.0 PORTFOLIO WEBSITE

Website Link: [<https://amidu-dabor.github.io/>]

4.0 CONCLUSION

This project reinforced key concepts of deep learning and demonstrated a full machine learning pipeline from data exploration to deployment readiness. Through model implementation, hyperparameter tuning, and the introduction of early stopping, we ensured model generalizability and robustness.

5.0 REFERENCES

1. TensorFlow Documentation: <https://www.tensorflow.org/>
2. Keras API Guide: <https://keras.io/api/>
3. Matplotlib Documentation: <https://matplotlib.org/>
4. Scikit-learn Docs (Classification): <https://scikit-learn.org/>
5. Seaborn Docs: <https://seaborn.pydata.org/>